Department of Computer Science
CS403NOD – Node.js

Lab Assignment – 08

Integrated Lab Server: All Previous Labs in One Server (Local + Live)

Semester: BCA VII

Lab No.: 08

Objective

- Bring all previous labs (Lab 01 to Lab 07) together in ONE Node.js server – without rewriting them
- Reuse the existing lab code from your NodeJS-Lab repository using module.exports and require()
- Show every lab's source code, live output and screenshots on a single portal website
- Run the integrated server locally and test every lab
- Deploy the same server live on Render (free tier) and show the same demo on the live URL
- Submit the live URL, GitHub link and screenshots of local and live runs

Important Note

You have already created each lab in its own folder and committed it to GitHub. You do NOT have to write those labs again. In this lab you only (1) make a very small change to the old lab files so they can be imported, and (2) build a new Lab-08 server that connects them.

How the Integrated Server Works

```
Browser -> Lab-08/server.js
  /labs/01/app/...  -> runs Lab 01 server (handler)
  /labs/03/run      -> runs Lab 03 script, shows output
  /labs/01          -> source code + screenshots
  /api/dashboard    -> labs, screenshots, requests
```

Two kinds of old labs:

- Server labs (Lab 01 simple server, Lab 02 student directory): their request-handling function is exported and Lab-08 calls it under /labs/<id>/app/...
- Script labs (Lab 03 async, Lab 04 modules/npm, Lab 06 fs, Lab 07 EventEmitter): Lab-08 runs the file with child_process and shows the printed output at /labs/<id>/run

Tasks

Task 1: Check Your Repository Structure

Your repository should already look like this (your folder names may be different – use your own names everywhere below). Create only the new Lab-08 folder.

```
NodeJS-Lab/
├── Lab-01-Simple-Server/server.js
├── Lab-02-Student-Directory/server.js
├── Lab-03-Async-Food-Delivery/food.js
├── Lab-04-Modules-npm/app.js
├── Lab-06-File-System/fs-demo.js
├── Lab-07-EventEmitter/emitter.js
└── Lab-08/                    (NEW)
    ├── server.js
    ├── labs.js
    ├── package.json
```

```
├── .gitignore
├── README.md
├── modules/
│   └── logger.js
└── public/
    └── screenshots/
```

Inside Lab-08 run:

```
npm init -y
npm install slugify
```

In package.json set "main": "server.js" and add the script:

```
"scripts": { "start": "node server.js" }
```

.gitignore of Lab-08 must contain:

```
node_modules/
logs/
.env
```

Task 2: Make Your Old Server Labs Importable (2-line change)

Open the server file of Lab 01 and Lab 02. Your code probably looks like this:

```
http.createServer((req, res) => {
  ... your routes ...
}).listen(3000);
```

Change it so that the request function has a name, is exported, and the server starts only when the file is run directly:

```
function handler(req, res) {
  ... your routes (same code, do not change) ...
}

module.exports = handler;                    // line 1: export it

if (require.main === module) {               // line 2: start only when run directly
  http.createServer(handler).listen(3000, () => console.log('Lab 01 running on 3000'));
}
```

Test that the old lab still works exactly as before:

```
node server.js        (run inside the old lab folder – it must still start on its own port)
```

Script labs (async, modules/npm, fs, EventEmitter) need NO change. They must simply run with node filename.js and print their output with console.log.

Task 3: Lab Registry (labs.js)

Create Lab-08/labs.js. It is a list with one entry for each old lab. Use your own folder and file names.

```
module.exports = [
  { id: '01', title: 'Simple HTTP Server', topic: 'http module, routing',
    type: 'server', file: '../Lab-01-Simple-Server/server.js', try: '/' },
  { id: '02', title: 'Student Directory API', topic: 'JSON, query parameters',
    type: 'server', file: '../Lab-02-Student-Directory/server.js', try: '/students?course=BCA' },
  { id: '03', title: 'Async JS Food Delivery', topic: 'Promise, async/await',
    type: 'script', file: '../Lab-03-Async-Food-Delivery/food.js' },
  // add Lab 04, Lab 06, Lab 07 (and any other lab you made) in the same way
];
```

id = lab number, type = 'server' or 'script', file = path of the old lab file relative to Lab-08, try = a sample URL to open (server labs only).

Task 4: Logger Module using EventEmitter (modules/logger.js)

- Create a class Logger that extends EventEmitter and export one instance of it

- Listen for the event 'request' with (method, url); print the line in console and append it to logs/server.log using fs.appendFile
- Create the logs folder automatically with fs.mkdirSync(..., { recursive: true })
- Add a listener for 'error' so a logging problem never crashes the server
- Export logger and the LOG_FILE path

Hint: a log line looks like 2026-10-06T07:22:20.499Z GET /labs/01

Task 5: The Integrated Server (server.js)

Create the server with http.createServer and an async handler. Port must come from the environment so the same file works locally and on Render, and it must listen on 0.0.0.0:

```
const PORT = process.env.PORT || 3000;
server.listen(PORT, '0.0.0.0', () => console.log(`Server running on port ${PORT}`));
```

Parse the URL with url.parse(req.url, true). For every request emit the 'request' event of your logger. Implement these routes:

Route	What it must do
GET /	Portal page (HTML): shows whether you are running 'local' or 'live (Render)' and a list of all labs with links to /labs/<id>
GET /about	JSON with project name, your name (process.env.STUDENT_NAME) and the number of labs
GET /health	JSON { status, environment, uptimeSeconds }. environment is 'live (Render)' when process.env.RENDER exists, otherwise 'local'
GET /labs	JSON list of all labs from labs.js; add a 'slug' for each title using the slugify package
GET /labs/:id	HTML page of one lab: title, topic, a link to run/open it, the complete SOURCE CODE of the lab (read with fs.promises.readFile, escape < > & before showing) and its screenshots
GET /labs/:id/run	Script labs: run the lab file with child_process.execFile(process.execPath, [file], { timeout: 5000 }) and return { ok, output, error } as JSON
GET /labs/:id/app/...	Server labs: remove the /labs/<id>/app part from req.url (keep the query string) and call the lab's exported handler: require(file)(req, res)
GET /screenshots/<name>	Send an image from public/screenshots using fs.createReadStream. Use path.basename() so nobody can read other files. Only .png / .jpg
GET /api/dashboard	Use Promise.all to read the log file and the screenshots folder together; return { labs, screenshots, requestsLogged }
Any other route	404 JSON { error: 'Route not found', path }

Rules:

- Create one helper send(res, status, data) used by every route
- Unknown lab id gives 404; calling /run on a server lab (or /app on a script lab) gives 400 with a helpful message
- Whole handler inside try/catch; unexpected errors return 500 { error: 'Internal Server Error' } and the server must never crash
- Run only the files listed in labs.js. Never run a file name taken directly from the URL

Task 6: Run Locally and Take Screenshots

```
cd Lab-08
npm start
```

Open and test each URL (use your own lab numbers):

```
http://localhost:3000/
http://localhost:3000/health
```

http://localhost:3000/labs
http://localhost:3000/labs/01 (source code + screenshots)
http://localhost:3000/labs/01/app/ (Lab 01 server working)
http://localhost:3000/labs/02/app/students?course=BCA
http://localhost:3000/labs/03/run (async output)
http://localhost:3000/labs/04/run
http://localhost:3000/labs/06/run
http://localhost:3000/labs/07/run
http://localhost:3000/api/dashboard
http://localhost:3000/labs/99 (must give 404)

Take a screenshot of the browser for every lab (address bar must be visible). Save them in Lab-08/public/screenshots using the naming rule:

lab01-local.png lab02-local.png lab03-local.png ... (local run)
lab01-live.png lab02-live.png lab03-live.png ... (live run – Task 8)

The file name must start with lab + the lab id, so the server shows it automatically on that lab's page.

Task 7: Push to GitHub

Your repository already contains the old labs. Commit the small changes in Lab 01 / Lab 02 and the new Lab-08 folder. The repository must be public so Render can read it. See GitHub Submission below.

Task 8: Deploy Live on Render (free)

Render must get the WHOLE repository (not only Lab-08) because Lab-08 reads the old lab folders. So leave Root Directory blank.

1. Go to render.com and sign up with your GitHub account
2. Click New → Web Service and select your NodeJS-Lab repository
3. Language / Runtime: Node. Root Directory: leave blank
4. Build Command: copy the command given below (it installs the packages of every lab that has a package.json)
5. Start Command: node Lab-08/server.js
6. Instance Type: Free
7. Optional environment variable: STUDENT_NAME = your name
8. Click Create Web Service and wait until the log shows the service is live
9. Open https://<your-service>.onrender.com and repeat ALL tests of Task 6 on the live URL
10. Take screenshots of every lab on the live URL (lab01-live.png ...), add them to Lab-08/public/screenshots, then git add / commit / push – Render redeploys automatically

Build Command:

```
for d in */; do if [ -f "$d/package.json" ]; then (cd "$d" && npm install); fi; done
```

If your repository has the labs inside a parent folder (for example NodeJS/NodeJS-Lab), set Root Directory to that parent folder and write the Start Command relative to it.

Important points about the free plan

- The service sleeps after 15 minutes without traffic; the first request afterwards can take about a minute – just wait and refresh. Open the live URL a few minutes before your demo.
- The file system is temporary. logs/server.log and any file created by an old lab (for example the file-system lab) is reset on every restart or redeploy. This is normal.
- The server must listen on process.env.PORT, otherwise deployment fails.
- Every git push redeploys the service automatically.

README.md

Create README.md inside Lab-08 with:

- Project title, your name, roll number, class
- Short description: what the integrated server does
- List of labs included (table: lab number, topic, type)
- Route table (copy from Task 5)
- Technologies used (http, fs, events, child_process, path, url, slugify)
- How to run locally (npm install, npm start)
- Live URL
- Screenshots (local and live) or the link to the screenshots folder
- What I learned (3–4 lines)

GitHub Submission

Inside your NodeJS-Lab repository run:

git add .
git commit -m "Lab 08 Submission - Integrated Lab Server"
git push
Check on github.com that Lab-08 is visible, the screenshots are uploaded, and node_modules is NOT uploaded.

Google Classroom Submission

- GitHub repository link
- Live deployed URL (Render link)
- Screenshots of every lab running locally (localhost)
- Screenshots of every lab running on the live URL

Evaluation Criteria

- Old labs made importable, labs.js registry correct – 3 marks
- Portal routes, source viewer and screenshots working – 3 marks
- Server labs and script labs both integrated – 3 marks
- EventEmitter logger, fs / async usage and error handling – 2 marks
- GitHub push, README and screenshots – 2 marks
- Live deployment working – 2 marks

Total: 15 marks

If You Face Any Problem

Do not copy someone else's code. Read the error message first, then ask your teacher with your question written clearly. Example:

"Sir, /labs/01/app/ shows Cannot GET / and the console says handler is not a function. In Lab 01 I exported the server, not the handler."
Common problems:

- handler is not a function – in the old lab, export the FUNCTION (module.exports = handler), not http.createServer(...)
- Old lab starts its own server when imported – you forgot if (require.main === module)
- Cannot find module – run npm install inside that lab folder (and check the path in labs.js)
- EADDRINUSE – port is already used; stop the old server with Ctrl + C
- Render: Application failed to respond – use process.env.PORT and listen on 0.0.0.0
- Render: lab page says file not found – Root Directory must be blank (or the parent of all labs), and paths in labs.js must match your real folder names

- Live URL is slow at first – the free service was sleeping, wait about one minute